Dynamic Pricing Using RL

What is Q-Learning

Earlier we have seen that agents had hardcoded rules: TFT always mirrors, Nash always plays cost. They never learn anything. Q-learning is the first agent that learns from experience. It starts knowing nothing and figures out a good pricing strategy purely by playing rounds and observing what happened.

The core idea is to maintain a table (the Q-table) that answers the question: "If the market is currently in state s, and I take action a, how much total future reward can I expect?"

That expected value is written Q(s, a). After enough experience, the agent just picks whichever action has the highest Q value for the current state.

The Bellman Equation

The Q-value update rule, derived from the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{Q(s',a')} Q(s,a) \right]$$

$$r + \gamma \max_{Q(s',a')} = TD\ target$$

Symbol	Meaning	In code it means
s	Current State	Last round's two prices (normalised)
a	Action Taken	The price index the agent chose
r	Reward received	Profit earned this round
s'	Next state	New prices after both firms moved
α	Learning rate	How fast to update (typically 0.1-0.3)
γ	Discount factor	How much future profit matters vs now (0.95)
$\max'_{\alpha} Q(s', a')$	Best possible future Q	Best Q-value achievable from next state

The bracket [TD target - Q(s,a)] is the TD error, the gap between what you expected and what you actually got plus future potential. We nudge Q(s,a) toward the target a little bit each update. Over thousands of rounds, Q-values converge to their true values.

ε-greedy exploration

Early in training, the agent knows nothing, its Q-table is all zeros. If it only exploits (picks max Q), it'll get stuck on the first thing that worked. So we use ε-greedy:

- With probability ε : pick a random action (explore)
- With probability 1- ε : pick the action with highest Q(s,a) (exploit)

Starting with ε = 1.0 (pure exploration) and decaying slowly to 0.05 over 80% of training. Too-fast decay = agent stops exploring before it's seen enough of the state space = gets stuck at a suboptimal price = fails the mid-project review gate.

State space design for your project

Our state is "last round's two prices." With 50 price levels per firm, the full state space is 50 x 50 = 2500 possible states. Our Q-table per agent is then:

$$Q\ table\ size = 2500\ states \times 50\ actions = 125000\ entries$$

Reward scaling

Our profit can be ~1012 at the monopoly price. Raw profits this large make Q-values explode to thousands, which causes numerical instability. So fixing: divide reward by the maximum possible profit before storing it, so rewards live in [0, 1]

$$r_{scaled} = \frac{\pi_i}{\pi_{\max}}\ where\ \pi \max\ \ = \frac{(a-bc)^2}{4b}$$

That denominator is the monopoly profit derived by plugging P* = $\frac{a+bc}{2b}$ back into the profit function.

Reward scaling — reward_scale=pi_max

Raw profit (up to 2025) gets divided by pi_max=2025 before entering the Q-table. So all rewards live in [0,1], Q-values stay bounded, no explosions.

Epsilon decay — epsilon_decay_frac=0.80

Epsilon goes from 1.0 → 0.05 over the first 80% of episodes (16,000 out of 20,000), then stays at 0.05. The agent explores heavily early, then gradually exploits what it learned.